

プログラミング入門2

第8回

基本型、文字列

篠埜 功

今日の内容

- 基本型、文字列について

基本型

- これまでに紹介した基本型はint型、double型である。その他にもさまざまな基本型があり、整数型、浮動小数点型に分類できる。

整数型

整数型は、char型, short int型, int型, long int型, long long int型の5種類ある。(C99でlong long int型が加わった)。char型が一番小さく、long int型が一番大きい(具体的に何バイトかは定められておらず、処理系によって異なる)。

それぞれの型は、符号付きと符号無しでそれぞれ2つずつある。型指定子(signedかunsigned)で指定する。char型以外は、型指定子を与えない場合は符号付きになる。char型は、型指定子を与えない場合は、符号付きか符号無しかのいずれかである(処理系依存)。

扱う数の範囲に応じて適切な型を選択して用いる。

char型が符号付きかどうか判定 (打ち込んで確認)

```
#include <stdio.h>
#include <limits.h>

int main (void) {
    printf ("この処理系のchar型は");
    if (CHAR_MIN)
        printf ("符号付きです。¥n");
    else
        printf ("符号無しです。¥n");
    return 0;
}
```

整数型

整数型は、結局、

- signed char型
- unsigned char型
- signed short int型
- unsigned short int型
- signed int型
- unsigned int 型
- signed long int型
- unsigned long int型
- signed long long int型
- unsigned long long int型

の10個となる。

符号について(1)

符号無し整数型(unsigned char, unsigned short int, unsigned int, unsigned long int, unsigned long long int)においては、ビット列を普通の2進数として解釈する。

符号について(2)

符号付き整数型(signed char, signed short int, signed int, signed long int, signed long long int)においては、符号を表すために1ビットを用いる(これを符号ビットという)。符号ビットが0の場合には、残りのビットを普通の2進数として解釈する。符号ビットが1の場合の解釈は以下の3通りのいずれかである(処理系依存)。

- 符号ビットを0に変えた場合のビット列が表す数にマイナスを付けた数を表す
- 2の補数表現
- 1の補数表現

0での除算について

CPUで整数同士で0での除算が実行されると、CPUが例外(exception)を発生させる。例外が発生する場合の動作はC言語の規格において未定義である。

(補足) Linuxにおいて、CPUで例外が発生した場合、制御がLinuxのkernelに移り、0除算を発生させたプログラムのプロセスにSIGFPEというsignalを投げ、signal handlerによってプログラムのプロセスが終了するのがdefaultの挙動である。詳しくはオペレーティングシステムの授業で勉強してください。

オーバーフロー(overflow)について

整数型で加算や乗算を行ったときに表現可能な値の範囲を超えた場合(オーバーフロー)、例外が発生する。この場合、C言語の規格で動作は未定義である。例外が発生した場合の通常の動作は0での除算と同様。

ただし、符号無し整数型の演算では、表現可能な値の範囲を超えた場合、表現可能な最大値+1で割った余りになるとCの規格で定められている(のでオーバーフローは発生しない)。

文字について

プログラム中では、1文字をクォートで囲むと、その文字に対応するint型の数を表す。

(例) 'a' はaという文字に対応するint型の数を表す。
(char型ではない。C++ではchar型だが。)演習室の環境では、'a'はint型の97を表す。

(補足) 文字列はchar型の並びである(後述)。

printfでの文字の表示方法

printf関数で文字を表示する場合、変換指定子を%cにする。引数にはint型を受け取り、それをunsigned char型に変換(256で割った非負の余りに変換)し、それに対応する文字を表示する。

```
#include <stdio.h>
int main (void) {
    printf ("%c\n", 'a');
    printf ("%d\n", 'a');
    printf ("%c\n", 97);
    printf ("%c\n", 97 + 256);
    printf ("%c\n", 97 - 256);
    return 0;
}
```

a

97

a

a

a

が表示される。

文字の8進表記

ある文字に対応する数が分かっているとき、その数で直接書きたい場合がある。その時に使うのが文字の8進表記である。例えば、'a'と書く代わりに(演習室の環境では)'¥141'と書くことができる。ただし、プログラムの可搬性が低下するので使わない方がよい。

よく使うのは、ヌル文字(null character、値は0、ナル文字と読んでもよい)を表す場合で、'¥0'と書く。'¥0'は、対応する数が0である文字に対応する数、つまり0を表している。0と直接書いても同じ意味だが、0よりも'¥0'の方が、文字を表している数だということが見た目に分かりやすいのでよく使う。

8進逆斜線表記の構文(参考)

8進逆斜線表記の構文1

¥ octal-digit

8進逆斜線表記の構文2

¥ octal-digit octal-digit

8進逆斜線表記の構文3

¥ octal-digit octal-digit octal-digit

ただし、octal-digitは0 1 2 3 4 5 6 7のいずれかを表す。

つまり、バックスラッシュのあとに1桁から3桁の8進数を書いたものが8進逆斜線表記である。

8進逆斜線表記はクオートおよびダブルクオートの中でのみ用いる。8進逆斜線表記をクォートで囲んだとき、それは、その8進数の表している数(int型)を表す。つまり、文字を表すための数であるということを見た目に分かりやすくするために用意されている構文である。

変数への格納

```
#include <stdio.h>
int main (void) {
    int x;
    x = 'a';
    printf ("%c¥n", x);    a
    printf ("%d¥n", x);    97
    x = x+1;
    printf ("%c¥n", x);    b
    printf ("%d¥n", x);    98
    return 0;
}
```

文字はint型やchar型の変数に格納される。ただし、文字列はchar型の配列である(後述)。

1を足したものを%cで表示するとbが表示される。

整数型の範囲について (打ち込んで確認)

C言語の処理系は、limits.hにおいて、それぞれの型の最大値、最小値をマクロとして提供する。

```
#include <stdio.h>
#include <limits.h>
int main (void) {
    printf ("char: %dから%dまで¥n", CHAR_MIN, CHAR_MAX);
    printf ("short int: %dから%dまで¥n", SHRT_MIN, SHRT_MAX);
    printf ("int: %dから%dまで¥n", INT_MIN, INT_MAX);
    printf ("long int: %ldから%ldまで¥n", LONG_MIN, LONG_MAX);
    printf ("long long int: %lldから%lldまで¥n", LLONG_MIN, LLONG_MAX);
    return 0;
}
```

long int, long long intの変換指定子には%ld, %lldを用いる。

整数型のまとめ

- 整数型はある一定の範囲の整数を表現するための型である。
- 扱う数値が負にならないことが分かっているならば、符号無し of 型を使うと、同じビット数で、より大きな範囲の数を持つことができる。

浮動小数点型

浮動小数点型は小数を表すための型であり、

- float型
- double型
- long double型

の3つがある。この演習では説明しない。

文字列とは(1)

char型の配列中の任意の場所から'¥0'(ヌル文字)までを文字列と言う。

'L'	'i'	'n'	'¥0'	'u'	'x'
-----	-----	-----	------	-----	-----

'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
-----	-----	-----	------	-----	-----	------

'¥0'	'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
------	-----	-----	-----	------	-----	-----	------

これらはchar型の配列であるとする。('L'や'¥0'などはint型だが、char型に変換されていると思って読んでください。)char型の配列の中で、'¥0'が最後(だけ)にある部分を文字列という。

文字列とは(2)

'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
-----	-----	-----	------	-----	-----	------

この部分は文字列である。

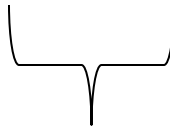
'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
-----	-----	-----	------	-----	-----	------

この部分も文字列である。

一番右端が'¥0'であり、それ以外に'¥0'を含んでいなければそれは文字列である。

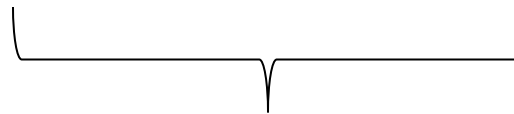
文字列とは(3)

'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
-----	-----	-----	------	-----	-----	------



この部分は文字列である(空文字列という)。

'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
-----	-----	-----	------	-----	-----	------



この部分も文字列である。

一番右端が'¥0'であり、それ以外に'¥0'を含んでいなければそれは文字列である。

char型の配列

文字列はchar型の配列を宣言し、各要素に文字を1文字ずつ格納して最後にヌル文字を格納することによって作成できる。

char型の配列型の変数は

```
char a [7];
```

のように宣言する。これはchar型の要素を7個持つ配列aを宣言している。この宣言下で

```
a[0] = 'L';
```

という代入文が実行されると、'L'(int型の数)が暗黙の型変換(第1回講義スライド参照)によりchar型に変換されてからa[0]に代入される。

printf関数での文字列の表示方法

```
#include <stdio.h>
int main (void) {
    char a [7];
    a[0] = 'L';
    a[1] = 'i';
    a[2] = 'n';
    a[3] = '¥0';
    a[4] = 'u';
    a[5] = 'x';
    a[6] = '¥0';
    printf ("%s¥n", &a[0]);
    return 0;
}
```

文字列の表示には、変換指定子に%sを用い、引数にchar型へのポインタを与える。そのポインタが指しているところから、ヌル文字の手前までを画面に表示する。

&a[0]は配列aの先頭要素へのポインタを表す。先頭要素は'L'であり、Linが表示される。

printf関数での文字列の表示方法 (打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    char a [7];
    a[0] = 'L';
    a[1] = 'i';
    a[2] = 'n';
    a[3] = '¥0';
    a[4] = 'u';
    a[5] = 'x';
    a[6] = '¥0';
    printf ("%s¥n", &a[1]);
    return 0;
}
```

&a[1]は配列aの2番目の要素へのポインタを表す。2番目の要素は'i'であり、inが画面に表示される。

注意

- 文字列の表示にはprintfの変換指定子として%sを使う。このとき、引数に与えられるポインタが指す先はchar型であることが前提なので、もしint型の配列に文字列を入れていた場合には正常に表示されなくなる。

間違った例

```
#include <stdio.h>
int main (void) {
    int a [7];
    a[0] = 'L';
    a[1] = 'i';
    a[2] = 'n';
    a[3] = '¥0';
    a[4] = 'u';
    a[5] = 'x';
    a[6] = '¥0';
    printf ("%s¥n", &a[0]);
    return 0;
}
```

このプログラムだと、意図した通りには表示されない。(具体的にどう表示されるかはint型の表現に依存。)

(さらに補足)

&a[0]はintへのポインタ型であり、char型へのポインタではないので、

\$ gcc -W -Wall test.c

のようにオプションを付けてコンパイルすればwarningが出る。

解説

この演習室の環境では、'L'はint型の76であり、
さきほどのint型の配列aは、メモリ上では

76	0	0	0	105	0	0	0	110	...
a[0]				a[1]				...	

のように並んでいる。すなわち、

'L'	'¥0'	'¥0'	'¥0'	'i'	'¥0'	'¥0'	'¥0'	'n'	...
-----	------	------	------	-----	------	------	------	-----	-----

ということである。つまり、この演習室の環境では'L'しか表示されなくなる。

ポインタについて

番地	100	101	102	103	104	105	106
	'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]

&a[0]は配列aの先頭要素へのポインタであり、&a[0]の値は、配列aの先頭要素の番地である。この場合、100である。&a[1]は配列aの2番目の要素へのポインタであり、101である。&a[2]は102である。以下同様。

printf関数における変換指定子の%sについて

番地	100	101	102	103	104	105	106
	'L'	'i'	'n'	'¥0'	'u'	'x'	'¥0'
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]

`printf ("%s", &a[1]);`
が実行されると、101番地から、ヌル文字の
1つ手前までの文字が画面に出力される。
つまり、inが表示される。

`printf ("%s", &a[4]);`
だと、uxが表示される。

補足

printf関数において変換指定子として%cを使う場合、引数の型はint型と書いたが、char型でもよい。もしchar型の式が与えられた場合は、規定の実引数拡張(default argument promotion)により、自動的にint型に変換される(printfの引数の個数は可変なので)。これは本講義の範囲外とし、説明はしない。

printfの変換指定子にはfloat型用のものはない。float型の式がprintf関数の第2引数以降に書かれた場合は、double型に変換される。これも規定の実引数拡張(default argument promotion)。

改行文字

改行文字は'\n'で表す。

'\n'自体はint型の数だが、char型の変数やchar型の配列の要素に代入するときは自動的にchar型の数に変換されてから代入される(暗黙の型変換)。

文字列リテラル

文字の並びをダブルクォート " で囲んだものを文字列リテラルという。

(例) "abc¥n" など。

文字列リテラルはchar型の配列であり、末尾にヌル文字'¥0'が追加されたものである。

(例) "abc¥n" は文字列リテラルであり、最後にヌル文字'¥0'が追加されるので、以下のような長さ5のchar型の配列である。

'a'	'b'	'c'	'¥n'	'¥0'
-----	-----	-----	------	------

文字列リテラルの連結

文字列リテラルは並べて書くことにより連結することができる。

```
#include <stdio.h>
int main (void) {
    printf("abc" "def" "ghijk" "¥n");
    return 0;
}
```

上記のprintfの部分は、
printf("abcdefghijk¥n");
と同じ意味である。

文字列リテラルの連結(続き)

(打ち込んで確認)

文字列リテラルを並べて書くとき、改行があってもよい。

```
#include <stdio.h>
int main (void) {
    printf("abc" "def"
           "ghijk" "¥n");
    return 0;
}
```

上記のprintfの部分は、
printf("abcdefghijk¥n");
と同じ意味である。

文字列リテラル (途中にヌル文字がある場合)

(例) "abc¥0de"は文字列リテラルである。最後にヌル文字が追加されるので、以下のような長さ7のchar型の配列である。

'a'	'b'	'c'	'¥0'	'd'	'e'	'¥0'
-----	-----	-----	------	-----	-----	------

例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    printf ("abc¥0de");
    return 0;
}
```

これを実行すると、abcが表示される(deは表示されない)。

文字列リテラル(空文字列)

(例) ""は文字列リテラルである。ヌル文字が追加されるので、以下のような長さ1のchar型の配列である。つまり、空文字列である。

' '

文字列リテラルの評価(参考)

文字列リテラルは一つの式であり、評価すると値が得られる。文字列リテラルの評価結果は、文字列リテラルが格納されている配列の先頭要素へのポインタである。

イメージ

例えば、“abc”という文字列リテラルが100番地から103番地に格納されていたとする。すると、“abc”の評価結果は100となる。

番地	100	101	102	103
	'a'	'b'	'c'	'\0'

pがcharへのポインタ型の変数の場合、p="abc"のような代入式が書ける。

文字列リテラルの記憶域期間(参考)

文字列リテラル(を格納している配列)は、プログラムの実行の開始から終了まで存在する。このことを、「文字列リテラルは静的記憶域期間(static storage duration)を持つ」という。

これまでのプロ入2の授業の例で出てきた、関数の中の複合文(ブロック)で宣言される変数は、プログラムの制御が宣言を通過するときにオブジェクト(箱)が生成され、その宣言が属している一番内側の複合文(ブロック)を抜けるときにそのオブジェクト(箱)は破棄される。このことを、自動記憶域期間(automatic storage duration)を持つという。

プロ入2の授業では説明しないが、関数の外で宣言される変数は静的記憶域期間(static storage duration)を持つ。

詳しくは教科書6-3節を読んでください。

文字列リテラルの書き換え(参考)

文字列リテラル(によって作られる配列)の各要素は書き換え不可。文字列リテラルの中身を書き換えるコードは動作が未定義。

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char *p;
    p="abc";
    p[0]='b'; /* 動作が未定義 */
    return 0;
}
```

p[0]='b'は文字列リテラルによって作られる配列の先頭要素を書き換えようとしており、動作が未定義。

```
#include <stdio.h>

int main(void) {
    char a[7] = "xyz";
    a[0]='w'; /* OK */
    return 0;
}
```

char a[7] = "xyz";は配列aがxyz¥0で初期化されるだけ。配列aはmain関数内で宣言されている配列で、各要素は書き換え可能。

文字の読み込み

1文字の読み込みは、scanf関数で変換指定子として%cを用いて行う。

```
#include <stdio.h>
int main (void) {
    char a;
    printf ("Input characters: ");
    scanf ("%c", &a);
    printf ("%c¥n", a);
    return 0;
}
```

%dや%sの場合と違い、%cの場合は入力文字が改行、空白、タブなどの場合も読み込まれる。

文字列の読み込み

文字列の読み込みは、scanf関数で変換指定子として%sを用いて行う。

```
#include <stdio.h>
int main (void) {
    char a[7];
    printf ("Input characters: ");
    scanf ("%s", &a[0]);
    printf ("%s¥n", &a[0]);
    return 0;
}
```

入力文字中に空白文字があると、空白文字の手前までが配列に格納される。

scanf関数における変換指定子の%sについて

番地	100	101	102	103	104	105	106
	'a'	'b'	'c'	'd'	'e'	'¥0'	
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]

```
scanf ("%s", &a[0]);
```

が実行されると、例えばabcdeをキーボードから入力した場合、100番地から104番地までa,b,c,d,eが順番に格納され、105番地にヌル文字が格納される。最後にヌル文字が格納されるので、配列のサイズ-1の長さを超えて入力してはいけない。それ以上入力したらあふれた部分も書きこまれる(これをバッファオーバーフローという)。サーバープログラム等でこのようなコードが用いられていると攻撃の対象になるので対処が必要だが、本講義の範囲外とする。

fgets関数

空白がある文字列はfgets関数を用いると読み込むことができる。fgets関数は読み込む文字数の上限を設定でき、上限の範囲内で、改行文字があったらそこまで読み、第1引数に与えられたポインタが指す先に書き込む。読み込んだ最後の文字(改行文字を読んだ場合は改行文字)の次にヌル文字を追加で書き込む。第2引数には、読み込む文字数の上限+1を与える。返り値は読み込んだ文字列の先頭文字へのポインタ(つまり第1引数のポインタと同じ)である。ただし、1文字も入力されなかった場合(いきなりEOFの場合)は何も行わずにNULLを返す。

(fgetsの使用例)

```
#include <stdio.h>
int main (void) {
    char a[10];
    fgets (a, sizeof(a), stdin);
    printf ("%s¥n", a);
    return 0;
}
```

strchr関数

文字列中のある文字が最初に出現する場所(ポインタ)を返し、ない場合はNULLを返す。string.hをincludeする必要がある。前ページの例で、最後に改行文字を読み込んだ場合にそれを消したい場合、以下のように書けばよい。

```
#include <stdio.h>
#include <string.h>
int main (void) {
    char a[10], *p;
    fgets (a, sizeof(a), stdin);
    p = strchr(a, '\n');
    if (p != NULL)
        *p = '\0';
    printf ("%s\n", a);
    return 0;
}
```

ダブルクォートやクォートの 表示等について

- ダブルクォートやクォート自体をダブルクォートやクォートの中に書きたい場合は、バックslashをその前につける。

例1

ダブルクォートやクォートを表示する例

```
#include <stdio.h>
int main (void) {
    printf ("¥'¥n");
    printf ("¥"¥n");
    printf ("%c¥n", '¥');
    printf ("%c¥n", '¥');
    return 0;
}
```

例2

ダブルクォートやクォートを変数に代入してから表示する例

```
#include <stdio.h>
int main (void) {
    int x,y;
    x = '¥';
    y = '¥';
    printf ("%c¥n", x);
    printf ("%c¥n", y);
    return 0;
}
```


基本課題1

英文をキーボードから入力し、その英文の長さを表示するプログラムを作成せよ。ただし、英字以外の文字(空白やピリオド等)も文字数にカウントする。fgets関数を使って良いが、参考課題1のように書いても良い。配列を使う場合は十分な長さの配列を宣言して使用せよ。

[実行例]

```
$ ./a.out
```

英文を入力して下さい: **This is a pen.**

あなたが入力した英文の文字数は14です。

```
$
```

基本課題2

英語の文字列(char *型)を受け取り、その中の空白の数を返す関数 countSpacesを定義せよ。

```
int countSpaces (char *str) { ... }
```

また、countSpacesが正常に動作することを以下のように確認せよ。
文字列をmain関数中においてキーボードから受け取り、それを関数 countSpacesに渡して戻り値として空白の数を受け取り、それをmain関数中で以下の実行例のように画面に表示する。

[実行例]

```
$ ./a.out
```

英語の文字列を入力してください: **I am a student.**

文字列"I am a student."中の空白の数は3個です。

(ヒント)空白(に対応するint型の値)は' ' (空白をクォート'で囲んだもの)で表される。

(注意) fgets関数を使って良いが、参考課題1のように書いても良い。(文字列格納用の配列は十分な長さで宣言せよ。)

(補足)関数countSpacesの仮引数のchar *strの部分はchar str []と書いても同じ意味である。

発展課題1

英文をキーボードから入力し、その英文中の単語の数を表示するプログラムを作成せよ。英字以外の文字(空白、ピリオド、コンマ、クエスチョンマーク、ハイフン、アポストロフィー等)は区切り文字とし、単語数にはカウントしないこととする。

[実行例]

```
% ./a.out
```

英文を入力してください: This is a pen.

単語数は4です。

```
% ./a.out
```

英文を入力してください: Is this a pen?

単語数は4です。

```
% ./a.out
```

英文を入力してください: How are you? --- I'm fine, thank you.

単語数は8です。

発展課題2

英文をキーボードから読み取り、英字の大文字だけを表示するプログラムを書け。

[実行例]

\$./a.out

文字列を入力: Unidentified Flying Object

UFO

\$

発展課題3

キーボードから英文を読み取り、それが回文かどうかを判定するプログラムを作成せよ。ただし、英字a-zの大文字小文字は区別せず、かつ英字以外の記号(空白、エクスクラメーションマーク等)は無視するものとする。

[実行例1]

英文を入力してください: **So many dynamos!**

"So many dynamos!"は回文です。

[実行例2]

英文を入力してください: **This is a pen.**

"This is a pen."は回文ではありません。

(参考) dynamoは発電機。dynamosはdynamoの複数形

(注意) 十分な長さの配列を宣言して用いること。

発展課題4

文字列を2つキーボードから読み取り、1つ目の文字列が2つ目の文字列の連続した部分文字列になっているかどうかを判定するプログラムを作成せよ。ただし、入力文字列中に空白やタブはないものとする。

[実行例1]

1つ目の文字列: **cde**

2つ目の文字列: **abcdef**

"cde"は"abcdef"の部分文字列です。

[実行例2]

1つ目の文字列: **cef**

2つ目の文字列: **abcdefg**

"cef"は"abcdefg"の部分文字列ではありません。

発展課題5

英語の文字列(char *型)を受け取り、その中の空白を全部削除する関数 deleteSpacesを定義せよ。

```
void deleteSpaces (char *str) { ... }
```

また、deleteSpacesが正常に動作することを以下のように確認せよ。

文字列をmain関数中においてキーボードから受け取り、関数deleteSpacesに渡し、その後main関数中で以下の実行例のように空白削除後の文字列を画面に表示する。

[実行例]

英語の文字列を入力してください: I am a student.
空白を削除すると"Iamastudent."になります。

(ヒント) 空白(に対応するint型の値)は ' ' (空白をクォート'で囲んだもの)で表される。

(補足) 関数deleteSpacesの仮引数のchar *strの部分はchar str []と書いても同じ意味である。

アルファベットの文字の判定

アルファベットの文字の大小の判定は、islower, isupper関数で行う。アルファベット文字かどうかは、islower, isupperのいずれも偽になるかどうかで判定できる。(あるいはisalpha関数を使ってもよい。)

```
#include <ctype.h>
#include <stdio.h>
void hantei (int c) {
    if (islower(c))
        printf ("%cは小文字です\n", c);
    else if (isupper (c))
        printf ("%cは大文字です\n", c);
    else
        printf ("%cはアルファベットではありません。\n", c);
}
```

```
/* 続き */
int main (void) {
    hantei ('a');
    hantei ('A');
    hantei ('+');
    return 0;
}
```


大文字から小文字への変換

大文字から小文字への変換はtolower関数を用いる。tolower関数は、int型を引数に受け取り、それが大文字に対応する数の場合、その小文字に対応するint型の数を返す。大文字以外の場合はそれをそのまま返す。ctype.hを読み込む必要がある。

```
#include <ctype.h>
#include <stdio.h>
int main (void) {
    printf ("%c¥n", tolower ('A'));
    printf ("%c¥n", tolower ('a'));
    printf ("%c¥n", tolower ('+'));
    return 0;
}
```

(補足)char型の配列の初期化について(1)

```
char a [1000] = {'¥0'};
```

```
char b [1000] = {'¥0'};
```

のようにchar配列の初期化をすることによって、すべての要素がヌル文字で初期化されるので便利がよい。(文字列のコピーなどの場合に最後のヌル文字の扱いがやりやすくなるので)

(補足)char型の配列の初期化について(2)

```
char a [10] = {'a', 'b', 'c', '¥0'};
```

のようにchar配列の初期化をすると、最初の3つがa,b,cで、残りが0で初期化される。この宣言は、

```
char a [10] = "abc";
```

と書くこともできる(上記の宣言と同じ意味)。

参考課題1

空白がある(かもしれない)文字列をキーボードから読み取り、表示するプログラムをfgets関数を使用せずに作成せよ。

[実行例]

文字列を入力してください: **This is a pen.**

入力した文字列はThis is a pen.です。

(ヒント)

配列を使って、scanfで変換指定子の%cを用いてwhile文の中で改行文字を読むまで1文字ずつ順番に配列の各要素に格納し、改行文字をヌル文字に置き換えればよい。その後、printfで変換指定子の%sを使って表示させればよい。(表示部分も1文字ずつwhile文で表示させることもできる。)

参考課題1 解答例

```
#include <stdio.h>
int main (void) {
    char s[100];
    int i;
    printf("文字列を入力してください: ");
    for (i=0; i<99; i=i+1) {
        scanf ("%c", &s[i]);
        if (s[i] == '¥n')
            break;
    }
    s[i] = '¥0';
    printf ("入力した文字列は%sです。¥n", &s[0]);
    return 0;
}
```

参考課題2

キーボードから英語の文字列を1つ読み取り、その中の英字の大文字をすべて小文字に変換したものを画面に表示するプログラムを作成せよ。英語の大文字以外の文字はそのまま表示せよ。

[実行例]

文字列を入力してください: **This is a pen.**

this is a pen.

(注意) 十分な長さの配列を宣言して用いること。

参考課題2 解答例

```
#include <stdio.h>
int main (void) {
    char s[100];
    int i;
    printf("文字列を入力してください: ");
    for (i=0; i<99; i=i+1) {
        scanf ("%c", &s[i]);
        if (s[i] == '¥n')
            break;
        if (isupper(s[i]))
            s[i] = tolower (s[i]);
    }
    s[i] = '¥0';
    printf ("%s¥n", &s[0]);
    return 0;
}
```